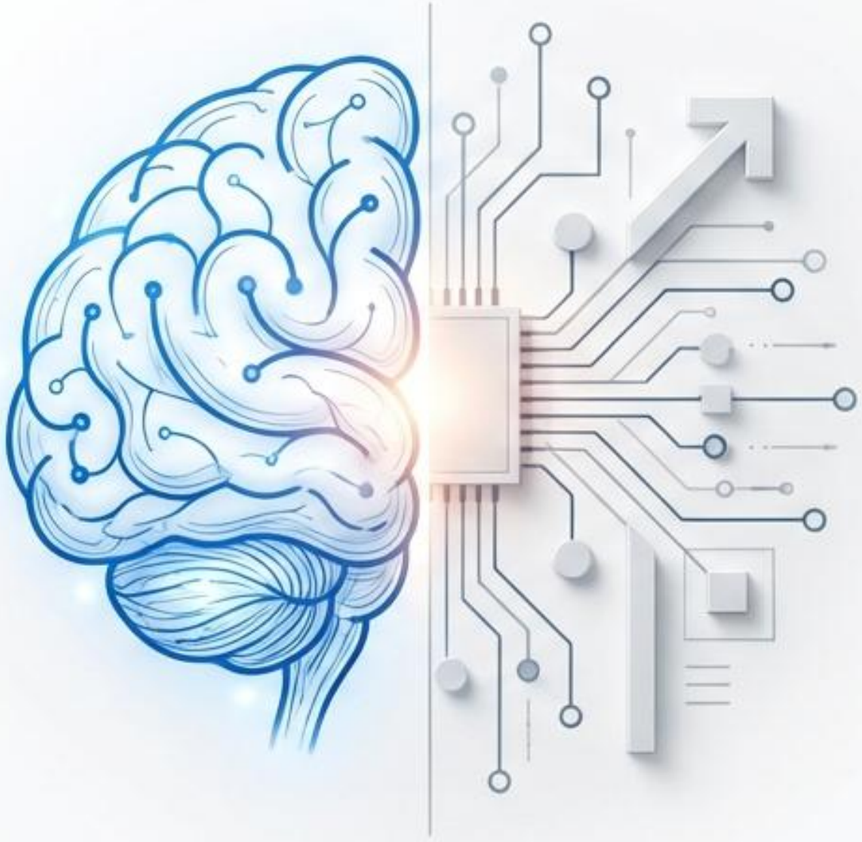


AI 시대, 왜 '프로그래밍 언어론'인가?

코딩의 종말이 아닌, 새로운 아키텍트의 탄생

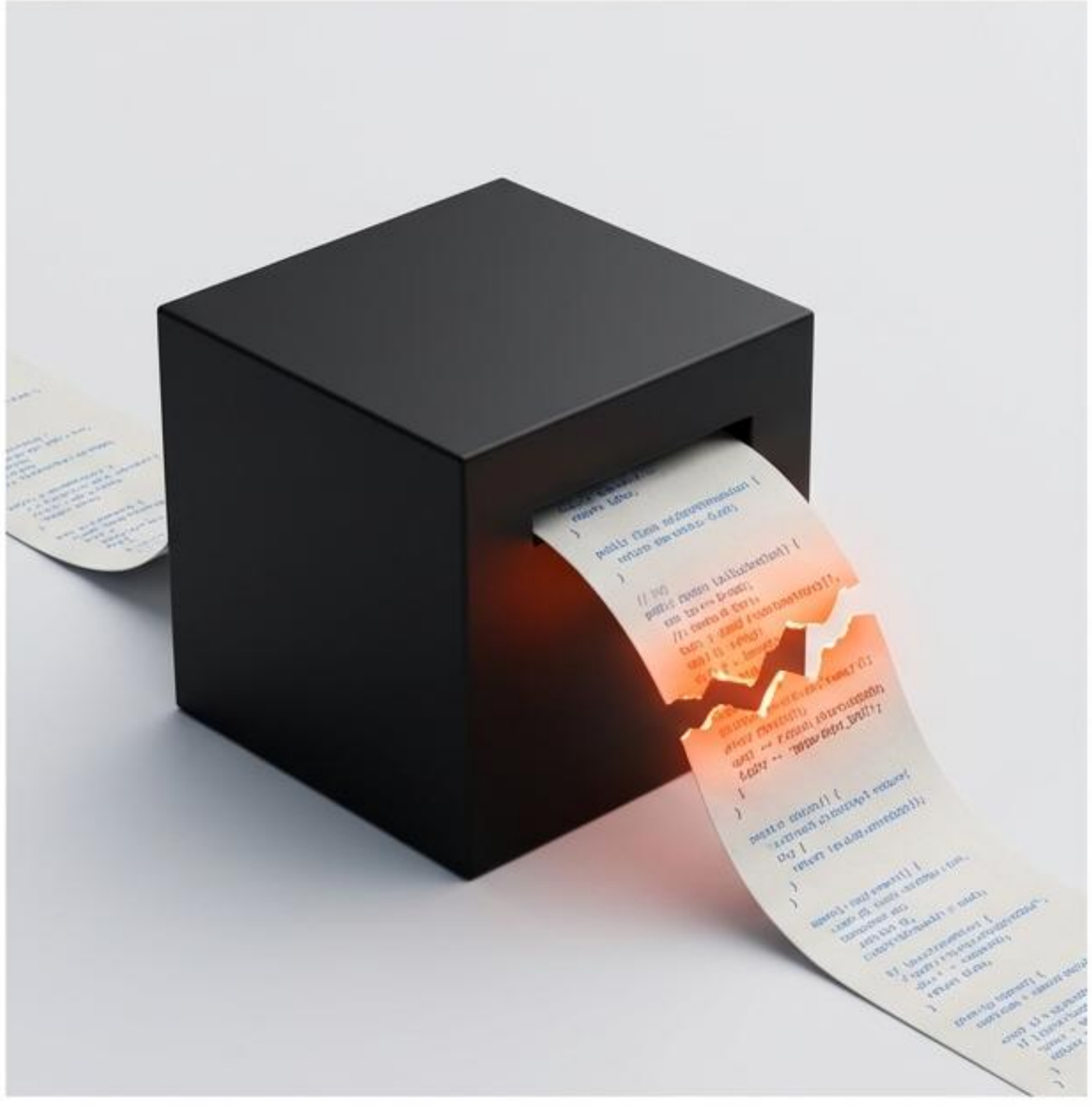
GitHub Copilot

Cursor

ChatGPT

코딩의 시대는 끝났는가?

* **현실 (Reality)::** GitHub Copilot, Cursor, ChatGPT의 등장
  - 복잡한 알고리즘의 즉각적 생성
* **의문 (Question)::** 문법 암기와 **메모리 구조 이해**의 불필요성 대두
* **진실 (Truth)::** '언어의 원리' 이해 여부가 개발자의 격차를 심화



AI 코딩의 '어두운 면' (The Black Box)

- **AI의 한계** (Limitations):
  - 확률적 코드 생성에 불과.
  - 논리적 안전성 및 효율성 보장 불가.
- **환각 현상** (Hallucination):
  - 존재하지 않는 라이브러리 사용.
  - 보안 취약점 포함 코드 제시.

역할의 대전환 (Role Shift)

Past

Future

Past

단순 '타이피스트 (Typist)'

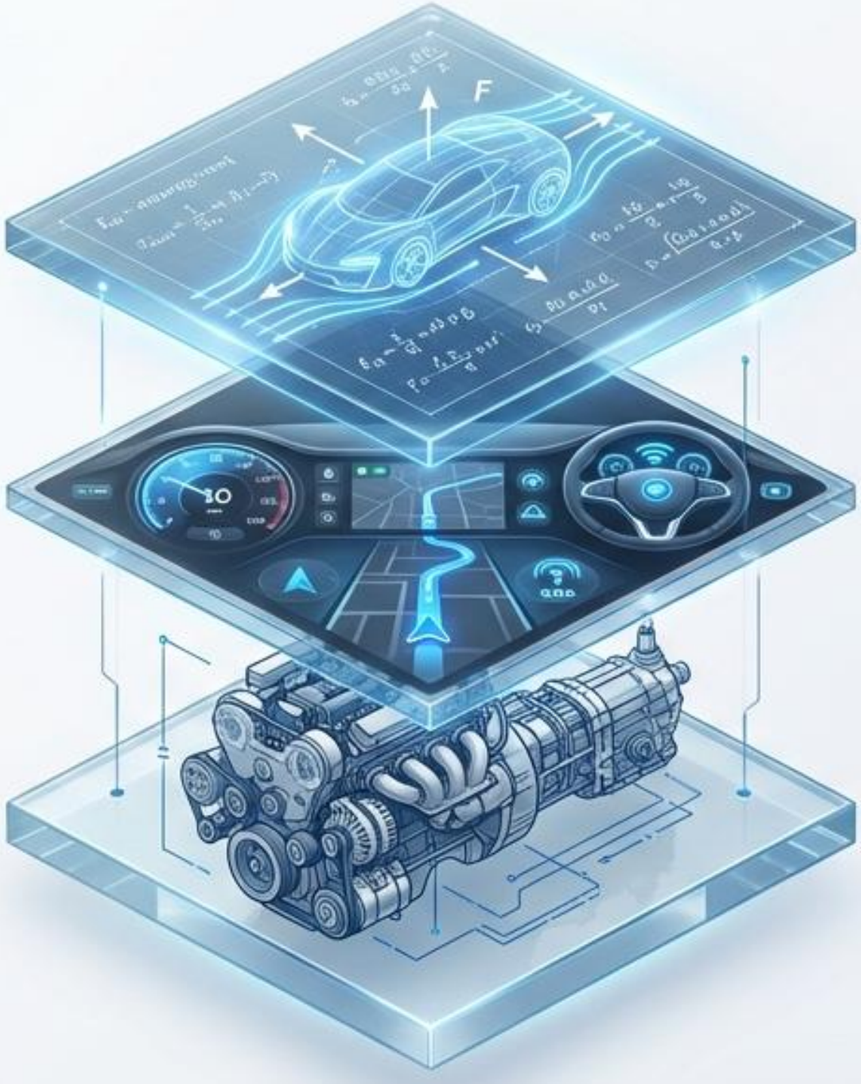
Future

코드 '검찰관' 및 '아키텍트 (Architect)'

Mission

AI가 생성한 코드의 검수 및 승인 권한 행사

C와 Python, 그 너머의 세계



언어론 (PL Theory): 도로의 물리 법칙과 에너지 효율 분석, 차세대 이동 수단 설계.

Python: 자율주행 자동차 운전 (자동 변속기).

C 언어: 자동차의 엔진 구조 이해 (수동 변속기).

언어는 곧 '생각의 틀'

> “내가 사용하는 언어의 한계가
> 내가 사는 세상의 한계다.”

— 루트비히 비트겐슈타인 (Ludwig Wittgenstein)

도구 종속 탈피 및 최적의 도구 선택 안목 배양

패러다임: 사고의 확장

명령형 (C/Python)

"이거 하고, 그다음 저거 해." (순차적 절차)

```
// C-style loop
for (int i = 0; i < 5; i++) {
    process(i);
    print("Step " + i);
}
```

함수형 (Haskell/Lisp)

데이터의 수학적 변환과 성질 정의.

```
-- Haskell map
map (\x -> x * 2) [1, 2, 3]
-- Result: [2, 4, 6]
```

안전 중심 (Rust)

메모리 소유권과 대여(Borrowing) 확인.

```
// Rust ownership
let s1 = String::from("hello");
let s2 = &s1; // Borrowing
// println:("{}", s1); // Valid
```

다양한 패러다임 습득 → AI에게 고차원적 명령 가능

이번 학기, 우리가 얻게 될 무기

1. 디버깅의 직관 (Debugging Intuition)

- 에러 메시지 해석 능력의 극대화.
- 즉각적 원인 파악 (예: 바인딩 시간 문제, 힙 메모리 오류).

2. 메타 학습 능력 (Meta-Learning)

- 새로운 언어의 '핵심' 파악 시간 단축 (3일 완성).
- 언어의 공통 원리를 통한 빠른 실무 적용.

결론: 도구의 주인이 되자!

역사적 교훈: 계산기 보급 ≠ 수학자의 소멸 (오히려 고등 수학 집중).

AI의 정의: 개발자를 위한 강력한 '계산기'.

최종 목표:

- 단순 '코더(Coder)' 탈피.
- 기술의 근본을 제어하는 '컴퓨터 과학자(Computer Scientist)'로 도약.